Desarrollo web: HTML, CSS y JavaScript

De la estructura semántica a la interactividad y el diseño adaptable

CódigoClaro · Material didáctico de apoyo · Edición octubre de 2026

Cómo estudiar este material

Lee el concepto, escribe los ejemplos sin copiar y modifica los casos de prueba. Los fragmentos son ejemplos didácticos: adapta nombres, tipos y validaciones al enunciado real.

1. HTML semántico

HTML describe la estructura y el significado del contenido. Utiliza encabezados jerárquicos, enlaces descriptivos, etiquetas de formulario asociadas y elementos semánticos como main, nav, article y footer.

```
<!doctype html>
<html lang="es">
<head><meta charset="utf-8"><title>Mi web</title></head>
<body>
  <main><h1>Catálogo</h1>
    <article><h2>Curso Java</h2><p>Empieza aquí.</p></article>
  </main>
</body>
</html>
```

2. Formularios accesibles

```
<label for="correo">Correo electrónico</label>
<input id="correo" name="correo" type="email" required>
<button type="submit">Enviar</button>
```

No dependas solo del placeholder. Añade mensajes de error claros. La validación del navegador mejora la experiencia, pero el servidor debe validar de nuevo.

3. CSS: cascada y caja

```
* { box-sizing: border-box; }
.card { padding: 1rem; border: 1px solid #ccc; margin-block: 1rem; }
.card h2 { color: #24345b; }
```

La cascada considera origen, importancia, especificidad y orden. El modelo de caja incluye contenido, padding, borde y margen. Prefiere clases reutilizables frente a selectores demasiado específicos.

4. Flexbox, Grid y responsive

```
.barra { display: flex; gap: 1rem; align-items: center; }
```

```
.catalogo { display: grid; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); gap: 1rem; }
img { max-width: 100%; height: auto; }
```

Flexbox es adecuado para distribuir elementos en una dimensión; Grid facilita filas y columnas. Usa media queries cuando el diseño lo necesite, prueba con teclado y no elimines el foco visible.

5. JavaScript moderno

```
const productos = [{ nombre: "Teclado", precio: 25 }];
const caros = productos.filter(p => p.precio > 20);
const nombres = caros.map(p => p.nombre);

async function cargar(url) {
  const respuesta = await fetch(url);
  if (!respuesta.ok) throw new Error("Error HTTP");
  return respuesta.json();
}
```

Usa const por defecto y let si reasignas. === compara sin coerción. filter crea una lista filtrada y map transforma elementos. Las operaciones asíncronas se gestionan con Promise y async/await; controla errores con try/catch.

6. DOM y eventos

```
const boton = document.querySelector("#saludar");
boton.addEventListener("click", () => {
  document.querySelector("#resultado").textContent = "¡Hola!";
});
```

Evita insertar datos no confiables mediante innerHTML. textContent es preferible para texto. Divide la lógica en funciones y evita variables globales innecesarias.

7. Mini proyecto

Construye un catálogo responsive de cursos con tarjetas, un filtro por tecnología y un formulario de contacto. Criterios: HTML semántico, etiquetas accesibles, CSS adaptable, validación básica y código JavaScript legible.

Checklist de revisión

- ¿Se puede navegar toda la página con teclado?
- ¿Las imágenes tienen texto alternativo apropiado?
- ¿La web se adapta a 360 px sin scroll horizontal?
- ¿Los datos se validan también en el servidor cuando existe backend?